CAGE — Presentation and Viva Script

Research Methodology, Ethics and Writing · Sessional 2

Divyansh Verma · Aniruddha Badhani · Ashutosh Singh

Part 1 — Presentation (about 10 minutes)

Lines in [brackets] are actions, not speech. Every number is a *scenario value* from our simulation and cost model; say "in our model" when you quote one.

Divyansh — Opening (30 s)

"Good morning. I'm Divyansh Verma, and with me are Aniruddha Badhani and Ashutosh Singh. [Hand over the printed paper.]

Our paper is *CAGE: When Does Communication-Graph Structure Pay Off for Rank Placement?*

In one line: we tested whether mapping 'who talks to whom' inside a distributed AI training job can make it faster. For most jobs the default setup is already right; it only helps when the default is wrong, and our paper shows how to tell the difference.

Aniruddha will explain the problem and our method, Ashutosh will present the results, and I'll close with the code."

Aniruddha — The problem (1 min)

"Large AI models are trained on many GPUs spread across several machines, and the GPUs send each other data at every step. Inside one machine that data moves fast; between machines it goes over the network and is much slower — about six times slower in our model.

Picture a team working in several rooms. People in the same room talk quickly; a message to another room takes time. So people who talk a lot should sit together. For GPUs, the rooms are machines."

Aniruddha — Original idea and how it changed (1.5 min)

"Our original idea was to record which GPU talks to which, draw it as a graph, and use the graph to put heavy talkers on the same machine. We wrote three hypotheses: H1, a high modularity score means the graph has real groups; H2, the busiest 20% of GPUs carry most of the traffic; H3, graph-based placement beats the default.

Before trusting them, we stress-tested our own plan. Tools that build these graphs already exist — MPIPP, TreeMatch, Scotch — so the idea itself is not new. We had compared against round-robin placement, which no real system uses; the real default is *block* placement, GPUs 0 to 7 on machine 1 and so on. And we had no cluster to measure speedups.

So we changed the question from ‘does it help?’ to ‘when does it help?’.”

Aniruddha — Method (1.5 min)

“Our method has five steps. One, record a trace: who sent how many bytes to whom. Two, break group operations like allreduce into the transfers they really make. Three, test whether the groups we find are real, by comparing against 20 random graphs with the same shape. Four, assign exactly eight GPUs per machine. Five, score each placement with a cost model, including the way NVIDIA’s NCCL library rebuilds its rings for each placement.

Every result is compared with block placement, and we wrote the expected result before running each of our seven experiments. All numbers come from simulation, not hardware, and the paper labels them as scenario values.

Ashutosh will now present the results.”

Ashutosh — Finding 1 (1 min)

“First, a high modularity score can be fake. A plain ring of 64 GPUs scores 0.74, well above the usual 0.3 threshold, but random graphs score just as high — its z-score is minus 1.1. Only stencils, Megatron and our planted test graph had real groups. Our second hypothesis also failed: the busiest 20% of GPUs carried at most 42% of the traffic, not 70 to 80%.”

Ashutosh — Finding 2, the main result (1.5 min)

“Against block placement, for the most common jobs — data-parallel training and Megatron in its normal order — the gain was zero. The default was already optimal, and our tool said ‘change nothing’.

Gains appeared only when the default order was wrong: scrambled GPU numbering cut traffic between machines by about 93%, stencils by 25 to 43%, and a mixture-of-experts model with spread-out groups by about 56%.

So graph-aware placement helps only when the default order disagrees with who actually talks to whom.”

Ashutosh — Two honest caveats (1 min)

“We report two results that went against us. First, the mixture-of-experts gain follows a setting we chose in our own simulation, and that case failed our significance test yet still gained — so we revised our rule: the test protects against fake groups but can miss real ones.

Second, when one machine’s network card slows down, the amount of data does not change, so a graph that only counts bytes finds the slow machine at chance level, 0.12. Weighting each transfer by its time finds it 75 to 100% of the time.”

Ashutosh — Limits and conclusion (1 min)

“Our limits: everything is simulated, and our trace readers have not yet been run on real logs. Real systems already handle the common cases, so we do not claim to have invented this. Our

contribution is a careful test of when graph-aware placement is worth it, including the parts of our own idea that failed. Next, we want real traces from multi-GPU jobs and a tool that flags badly placed jobs."

Divyansh — Demo and close (1 min)

"Everything in the paper can be checked. [Run `python -m pytest`.] These 38 tests check the code behind our numbers. [Run `cage run --paper 1 --only e2`.] This reproduces Table II in a few seconds; every number in the paper comes from a results file like this.

To summarise: we expected graphs to make distributed training faster. Testing it properly showed the default is already right for most jobs, and graphs help only when it is wrong. Our main lesson was methodological: test your own idea against the real default, and report what fails. Thank you — we're happy to take questions."

Part 2 — Viva answers

Answer in one or two sentences, then stop. If you do not know, say "we didn't test that; it's a limitation" — never invent a number.

Aniruddha — Sections I–III (problem, related work, design)

What is your research problem? "When does graph-aware placement beat the placement a launcher already produces?"

What is new, if MPIPP, TreeMatch and Scotch exist? "Not the partitioner. Our contribution is the testing protocol — null model, block baseline, NCCL regime — and the measurements, including negative results."

What is modularity? "How much more traffic falls inside groups than random chance would put there."

What is a null model? "Random graphs with the same degrees as ours. If our score is not clearly higher than theirs, the groups are not real."

Why 20 random graphs? "The smallest possible p-value is 1 over N plus 1. To reach 0.05 you need at least 19, so we used 20."

What is block placement? Round-robin? "Block puts consecutive ranks on one machine — the real default. Round-robin puts rank r on machine r mod H; nobody uses it, so it is a strawman."

What is the NCCL regime? "NCCL rebuilds its communication ring for each placement, so we re-expand collectives as host-ordered rings instead of keeping them fixed in rank order."

What does the cost model compute? "For each machine, the larger of data out or in over network bandwidth, plus internal data over 300 GB/s, plus a per-message cost; the job time is the slowest machine. It ranks placements; it does not predict seconds."

Why drop betweenness? “Messages do not pass through other ranks, so shortest paths have no physical meaning here.”

Ashutosh — Sections IV–VII (method, results, discussion, conclusion)

What type of research is this? “Empirical, quantitative, analytical and applied, using simulation and analytical modelling.”

Variables? “Independent: workload, placement, mixing, affinity, injected faults. Dependent: cross-machine bytes, modelled time, recovery scores, z, hit rate. Controlled: seeds, eight GPUs per machine, one cost model.”

Main result? “Zero gain for data-parallel and Megatron; gains of 25 to 93% only where the rank order is wrong.”

MoE gains 56% but is not significant — contradiction? “Yes, and we report it. The dense allreduce dominates the graph and hides the expert groups, so we revised the rule.”

Why does MoE with no affinity show 14.9% time reduction? “Moving ranks balanced load across network cards without removing traffic; we treat it as a side effect.”

Why does round-robin tie on MoE? “Its data groups are interleaved as r mod 8, exactly matching round-robin’s machines — a coincidence of the setup.”

Isn’t the MoE gain just your own setting? “Partly. It grows with the affinity we chose, which is why measuring real affinity from real traces is our first next step.”

Validity threats? “Internal: the cost model ignores overlap and congestion. External: synthetic workloads. Construct: cross-machine bytes are not the same as real speed.”

What would change your conclusion? “Real traces showing that common jobs are often misplaced.”

Divyansh — code, simulations and the change of idea

How were the experiments run? “Seven Python scripts with fixed seeds generate the workloads, run the analysis and write results files; `cage run --paper 1` reruns them all.”

How do you ensure reliability? “38 automated tests, fixed seeds, every number traced to a results file, and expected results written before each run.”

Why did the idea change? “Testing showed every original hypothesis failed against the realistic baseline, so we reframed the paper around when placement helps.”

Why no hardware? “No cluster access. Every number is labelled a scenario value, and the paper lists what real traces must show.”

Ethics? “We dropped a false novelty claim, used a realistic baseline, labelled model numbers, reported results that went against us, and made every number traceable.”

Did you use AI? [Answer truthfully, stating exactly what it was used for and how you checked its output.]

Numbers to remember

What	Value
Ring modularity	$Q = 0.74$, $z = -1.1$
Significance rule	$z > 3$ and $p \le 0.05$, 20 null graphs
Busiest 20% share	at most 0.42
Gains vs block	DP, Megatron 0%; scrambled Megatron −92.7%; 2D stencil −42.9%; 3D stencil −25.0%; MoE ($a = 0.6$) −56.3%
Slow NIC detection	bytes 0.12 (chance 0.125); time-weighted 0.75–1.00
Setup	64 GPUs, 8 per machine, 50 GB/s NIC, 300 GB/s intra-machine
Runtime	10.7 s at 1024 ranks